

Filière : Licence d'Excellence en Intelligence Artificielle

Projet de Fin d'Études du Module Deep Learning (PFM)

Système de détection des anomalies dans code VHDL

Liverable : rapport du projet

Membres d'équipe (groupe 2 /pfm) :

- 1. ATIK ZAKARIA
- 2. ILYASS BOUDADE
- 3. YOUSSEF DOUIBA

Supervisé par :

-El Habib Benlahmar
- Mme Salma

Table des chapitres :

Introduction	2
Imports and configuration.....	4
Chargement et Nettoyage Initial du jeu de données	5
Définition des fonctions	6
Génération des sous-ensembles de mutation.....	7
Tokenisation et compilation du vocabulaire	8
Enregistrement du dataset nettoyé.....	8
Encodage des tokens et standardisation des séquences	9
Distribution des types d'erreurs	10
Model Design.....	11
Evaluations	14
Conclusion	15

Introduction :

Avec l'évolution croissante des systèmes numériques, les langages de description matérielle tels que VHDL (*VHSIC Hardware Description Language*) jouent un rôle essentiel dans la conception et la simulation des circuits électroniques. Cependant, l'écriture du code VHDL peut être sujette à différents types d'erreurs, notamment des anomalies syntaxiques liées au non-respect des règles du langage, ainsi que des anomalies sémantiques qui affectent la logique et le comportement attendu du circuit.

La détection de ces anomalies constitue une étape importante afin de garantir la fiabilité et la qualité des conceptions matérielles. Les méthodes traditionnelles de vérification, telles que la compilation ou la simulation, permettent de détecter une partie des erreurs, mais elles peuvent être limitées lorsqu'il s'agit d'identifier automatiquement des comportements inhabituels ou des anomalies plus complexes.

Dans ce contexte, l'intelligence artificielle, et plus particulièrement les modèles d'apprentissage profond, offrent de nouvelles approches pour analyser le code source et reconnaître des motifs anormaux. Parmi ces modèles, RNN (*recurrent neural network*) sont particulièrement adaptés au traitement des données séquentielles avec une mémoire temporelle, comme le texte ou le code, grâce à leur capacité à mémoriser le contexte et les relations entre les éléments d'une séquence.

Ce projet présente la conception et la mise en œuvre d'un système de détection d'anomalies dans du code VHDL basé sur un modèle RNN. L'objectif du projet est de construire un modèle capable d'apprendre les structures normales du langage VHDL puis de détecter automatiquement les fragments de code présentant des erreurs syntaxiques ou sémantiques. Le rapport détaillera les différentes étapes du projet, notamment la préparation du jeu de données, le prétraitement du code VHDL, la conception de l'architecture du modèle RNN, son entraînement ainsi que l'évaluation de ses performances.

Dataset utilisé :

Ce dataset contient des fragments de code VHDL décrivant des composants matériels tels que des entités, architectures, processus et signaux.

- **Type** : dataset textuel basé sur du code structuré en JSON
- **Domaine** : conception matérielle (*Hardware Description Languages*)
- **Contenu** : exemples de code VHDL représentant des modules matériels
- **Objectif global du projet** : faciliter la compréhension, la traduction et la génération de code HDL à l'aide de modèles RNN

Le dataset permet au modèle d'apprendre :

- la structure correcte du langage VHDL
- les règles syntaxiques fondamentales (*pour détection des erreurs de grammaire, mots-clés mal utilisés ou structures VHDL invalides*)
- l'organisation logique d'un module matériel (*si le code est bien écrit mais l'objectif demandé est incorrect*)

⇒ le modèle RNN avec ce dataset va comprendre le comportement normal du code VHDL pour

contenu :

La colonne **prompt** est *input instruction or context* donné.

La colonne **chosen** contient la réponse préférée/correcte pour la **prompt**.

Extrait du dataset:

prompt	chosen
string · lengths	string · lengths
138 206k	39 205k
translate the following VHDL to verilog library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL...	module transceiver_clocking(input wire refclk0_p, input wire refclk0_n, input wire refclk1_p, input wir...
translate the following VHDL to verilog library ieee; use ieee.std_logic_1164.all; entity _...	module DE0_Nano_SoC_LT24_top_level(output wire ADC_CONVST, output wire ADC_SCK, output wire ADC_SDI,...
translate the following VHDL to verilog library IEEE; use IEEE.STD_LOGIC_1164.ALL; entity Mux4to1 is port (...)	module Mux4to1(input wire [3:0] DecHor, input wire [3:0] UniHor, input wire [3:0] DecMin, input wire...
translate the following VHDL to verilog library ieee; use ieee.std_logic_1164.all; entity ic4821 is port (d...	module ic4821(input wire [7:0] d, input wire pl, input wire ds, input wire cp, output wire q5, output...
translate the following VHDL to verilog library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.ALL...	module aux_interface(input wire clk, output reg [7:0] debug_pmod, inout wire dp_tx_aux_p, inout wire...
translate the following VHDL to verilog library IEEE; use IEEE.STD_LOGIC_1164.ALL; use...	module resolution_mouse_informer(input wire clk, input wire rst, input wire resolution, input wire...
< Previous 1 2 3 ... 87 Next >	

Imports and configuration :

Dans la 1^{ère} phase on va préparer l'environnement nécessaire à la création du jeu de données.

```
# préparation du dossier du dataset

DATA_DIR = "data"
if os.path.exists(DATA_DIR):
    shutil.rmtree(DATA_DIR)
os.makedirs(DATA_DIR, exist_ok=True)
```

Les bibliothèques Python dédiées au traitement de texte, à la manipulation des données qui sont :

- **re** : Nettoyer et découper le code VHDL en unités (*tokens*).
- **json** : Charger le dataset VHDL ou sauvegarder des données prétraitées
- **random** : générer des **valeurs aléatoires** ou d'effectuer des choix aléatoires
- **os** : Permet d'interagir avec le système d'exploitation, comme créer des dossiers, vérifier des chemins ou gérer des fichiers
- **shutil** : Créer et gérer les dossiers ainsi que les fichiers du projet
- **pickle** : Sauvegarder le tokenizer, le vocabulaire ou les objets de prétraitement.
- **numpy** : Manipuler les tableaux numériques utilisés comme entrées du réseau RNN.
- **pandas** : Organiser les échantillons de code VHDL et leurs labels dans des DataFrames.
- **Counter** : Compter la fréquence des tokens VHDL pour construire le vocabulaire du modèle.

Les opérations sur les fichiers et à la sauvegarde d'objets sont importées afin de fournir les outils nécessaires aux étapes suivantes. Un répertoire de travail dédié au dataset est ensuite créé après la suppression de toute version précédente, garantissant ainsi une exécution propre et reproductible du processus. La structure du dataset est définie en fixant un total de **8 613 échantillons VHDL**, répartis de manière équilibrée entre des exemples sains et des exemples contenant des anomalies, afin de limiter les biais lors de l'apprentissage du modèle RNN.

Enfin, une longueur maximale de séquence de **512 tokens** est définie pour normaliser la taille des entrées, permettant ainsi leur traitement efficace par le réseau de neurones récurrent.

Chargement et Nettoyage Initial du jeu de données

Cette phase consiste à charger puis nettoyer le jeu de données VHDL afin de préparer des entrées de qualité pour l'apprentissage du modèle RNN.

Le dataset est importé depuis la plateforme Hugging Face.

```
df = pd.read_json("https://huggingface.co/datasets/hdl2v/vhdl-dataset/resolve/main/vhdl_data.json")
```

Puis les instructions textuelles inutiles précédant le code VHDL sont supprimées pour ne conserver que le contenu technique pertinent.

```
df['code'] = df['prompt'].apply(
    lambda x: re.sub(r"translate the following VHDL to verilog\s*", "", str(x), flags=re.IGNORECASE)
)
```

La normalisation est ensuite appliquée en uniformisant les caractères en majuscules et en standardisant les retours à la ligne, ce qui permet de réduire la variabilité inutile dans les données. Les espaces et lignes vides excessifs sont également supprimés afin d'obtenir un format de code plus homogène.

```
df['code'] = df['code'].str.upper()
df['code'] = df['code'].str.replace('\r\n', '\n').str.replace('\r', '\n')
df['code'] = df['code'].apply(lambda x: re.sub(r'\n{3,}', '\n\n', x).strip())
```

Enfin, les exemples dupliqués ainsi que les colonnes non nécessaires sont éliminés afin d'obtenir un dataset propre et optimisé pour les prochaines étapes de traitement et d'entraînement du réseau de neurones récurrent.

```
df.drop_duplicates(subset=['code'], inplace=True)
df = df.drop(columns=['prompt', 'chosen']).reset_index(drop=True)
```

Définition des fonctions

- Tokenisation (bibliothèque utilisé à mentionner : re(regex))

Pour préparer les séquences VHDL pour leur exploitation par le modèle RNN et d'enrichir le jeu de données avec des exemples anormaux Une fonction de **tokenisation spécifique au langage VHDL** est mise en place afin de découper chaque fichier source en une suite de tokens représentant les mots-clés, identifiants, opérateurs, nombres et symboles importants du langage.

Les tokens sont ensuite normalisés en majuscules afin de réduire la taille du vocabulaire et d'assurer une représentation uniforme des instructions VHDL.

- Introduction d'erreurs sémantiques (bibliothèque utilisé à mentionner : Random - re(regex))

Une fonction de génération d'anomalies artificielles est utilisée pour transformer des codes VHDL corrects en versions erronées en détectant les blocs du «BEGIN » et « END » pour commencer.

```
def introduce_semantic_error(vhdl_code):
```

Cette génération repose sur plusieurs stratégies telles que la suppression d'éléments structurels essentiels du code, la désorganisation des instructions internes de l'architecture (*exemple : remplacement des opérations logiques AND et OR entre eux pour créer des erreurs sémantiques*), la corruption volontaire de la syntaxe (*exemple : changement des opérateur d'affection « => , <= » ou reorganisation des partie du code entre les blocs pour les erreurs syntaxiques*) et l'injection de lignes de code invalides ou inutiles.

Ces modifications permettent de créer un ensemble varié d'exemples anormaux, comprenant des erreurs syntaxiques et sémantiques, nécessaire pour entraîner le réseau de neurones récurrent à distinguer les codes VHDL valides des codes contenant des anomalies.

Si le code a été modifié avec succès, la fonction renvoie immédiatement le code VHDL corrompu et le type d'anomalie constatée.

Génération des sous-ensembles de mutation

Cette phase consiste à construire un jeu de données équilibré composé d'exemples normaux et d'exemples anormaux à partir des données VHDL initiales.

Une copie du dataset est d'abord créée afin de préserver les données originales et permettre la génération contrôlée de variations corrompues par la fonction que nous avons créée précédemment

```
# création d'une copy du dataset pour la création/gestion d'anomalie
df_mutation_pool = df.copy()
df_mutation_pool["original_idx"] = df.index
df_mutation_pool = df_mutation_pool.sample(frac=1, random_state=42)
```

Chaque échantillon est ensuite transformé à l'aide d'une fonction de mutation générant des anomalies syntaxiques et sémantiques, tout en conservant une trace du type d'erreur introduit.

```
mutated_outputs = df_mutation_pool['code'].apply(introduce_semantic_error)
```

Les exemples réellement corrompus sont filtrés et étiquetés comme anomalies, tandis qu'un sous-ensemble est sélectionné pour respecter la taille cible du jeu de données.

```
df_anomalies_candidates = df_mutation_pool[df_mutation_pool['error_type'] != "None"].copy()
```

En parallèle, les échantillons utilisés pour générer ces anomalies sont retirés du pool de données normales afin d'éviter toute fuite de données entre les classes.

```
# Suppression des échantillons utilisés pour des anomalies dans l'ensemble de données d'origine.
original_indices_used_for_anomalies = df_anomalies["original_idx"]
df_clean_pool = df.drop(original_indices_used_for_anomalies)
```

Enfin, un ensemble d'exemples sains est constitué et étiqueté comme non-anormaux, garantissant ainsi la création d'un dataset final équilibré

et cohérent, prêt pour l'entraînement du modèle de détection d'anomalies.

Tokenisation et compilation du vocabulaire

On doit transformer les données VHDL nettoyées en une représentation adaptée à l'apprentissage du modèle de détection d'anomalies.

Les codes VHDL des ensembles normal et anormal sont d'abord tokenisés afin de les convertir en séquences de symboles structurés.

Ces deux ensembles sont ensuite fusionnés pour former un jeu de données unique, qui est mélangé afin d'éviter tout biais lié à l'ordre des données.

Un compteur de fréquence est ensuite construit afin d'analyser la distribution des tokens présents dans l'ensemble du dataset.

Sur cette base, un vocabulaire est généré en intégrant des tokens spéciaux indispensables au traitement des séquences, tels que les symboles de padding, d'inconnu, de début et de fin de séquence.

Les tokens les plus fréquents sont ensuite ajoutés au vocabulaire avec un index unique, tandis que les tokens rares sont ignorés afin de réduire la complexité du modèle.

Enfin, ce vocabulaire est sauvegardé dans un fichier JSON pour garantir sa réutilisation lors des étapes d'entraînement et d'inférence du modèle RNN.

Enregistrement du dataset nettoyé

Pour maintenant, les nécessaires formations et ajustements des deux classes sont finies, on va exporter le jeu de données final dans un format CSV afin de le rendre facilement exploitable pour les étapes ultérieures du projet.

Une copie du dataset est d'abord créée afin de préserver les données originales intactes.

Les sauts de ligne présents dans le code VHDL sont ensuite supprimés pour garantir la compatibilité avec le format CSV, qui ne supporte pas correctement les données multi-lignes. De plus, les séquences de tokens sont transformées en chaînes de caractères sous forme de texte séparé par des espaces, ce qui permet de conserver l'information séquentielle tout en assurant une structure adaptée au stockage tabulaire.

A la fin, le dataset est sauvegardé dans un fichier CSV, facilitant ainsi son réutilisation pour l'entraînement, l'évaluation ou l'analyse du modèle de détection d'anomalies.

Encodage des tokens et standardisation des séquences

Maintenant la dataset est formée, on va transformer ses données textuelles et catégorielles en représentations numériques exploitables par le modèle de réseau de neurones récurrent.

Les différents types d'erreurs sont d'abord encodés sous forme de valeurs numériques afin de permettre une classification supervisée des anomalies.

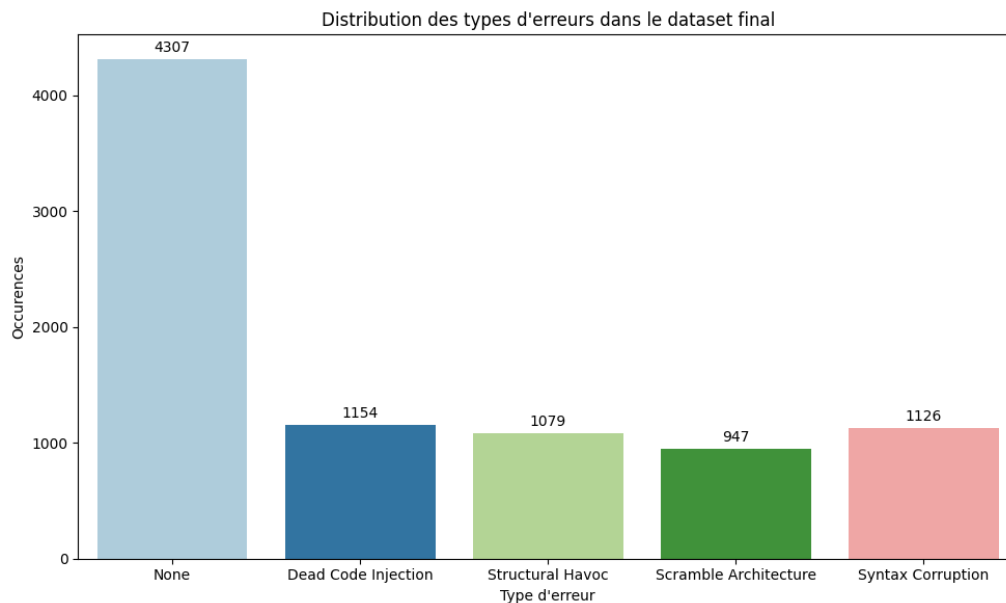
Une fonction d'encodage est ensuite définie pour convertir chaque séquence de tokens en une suite d'identifiants numériques basés sur le vocabulaire construit précédemment.

Chaque séquence est enrichie par des tokens spéciaux indiquant le début et la fin de la séquence, puis ajustée à une longueur fixe grâce au padding ou à la troncature afin d'assurer une compatibilité avec l'architecture du modèle.

Enfin, les données sont transformées en tenseurs numériques pour les entrées du modèle, tandis que deux types de labels sont générés : un label binaire indiquant la présence ou l'absence d'anomalie, et un label multiclassés représentant le type spécifique d'erreur.

Cette étape finalise la préparation des données pour l'entraînement du modèle de détection d'anomalies basé sur un réseau de neurones récurrent.

Distribution des types d'erreurs



- **4307** : normal
- **4306** : corrompues séparées en différentes catégories :
 - ⇒ **1154** : insertion des segments de code qui n'affectent pas le comportement final du circuit ou qui ne sont jamais exécutés/utilisés.

EX : signal temp : std_logic;

temp <= '1'; <- n'est pas utilisé

- ⇒ **1079** : la rupture de la structure logique des modules VHDL, comme une inadéquation entre l'entité et l'architecture ou une organisation hiérarchique incorrecte.

EX : entity counter is

end counter;

architecture wrong_arch of adder is -- nom d'entité incorrect

begin

end;

- ⇒ **947** : réarrangement ou au mélange d'instructions logiques à l'intérieur du bloc d'architecture de manière à rompre la logique d'exécution

⇒ 1126 : erreurs syntaxiques

Model Design

On va fixer les paramètres nécessaires à la reproductibilité des expériences et à la configuration du modèle RNN.

Un générateur aléatoire est initialisé avec une graine fixe afin de garantir la stabilité des résultats entre différentes exécutions.

Le vocabulaire préalablement construit est ensuite rechargé afin d'assurer la cohérence entre les étapes de prétraitement et d'entraînement.

Les principaux hyperparamètres du modèle sont définis, notamment la taille du vocabulaire, la dimension des embeddings, ainsi que la taille de l'état caché du réseau RNN.

Les paramètres d'apprentissage tels que le taux d'apprentissage, la taille des lots et le nombre d'époques sont également spécifiés afin de contrôler le processus d'entraînement.

```
VOCAB_SIZE = len(vocab)
EMBED_DIM = 64      # embedding dimension
HIDDEN_DIM = 128    # RNN hidden state size
N_BINARY = 2        # clean vs anomaly
N_MULTI = 4          # None / Inverted / Relational / Logical
LR = 3e-3           # learning rate (Adam)
BATCH_SIZE = 64
EPOCHS = 15
CLIP_NORM = 5.0      # gradient clipping threshold
```

Une technique de clipping des gradients est enfin introduite pour stabiliser l'apprentissage et éviter le problème d'explosion des gradients, particulièrement fréquent dans les architectures récurrentes. Cette étape permet ainsi de préparer un cadre d'entraînement stable, reproductible et bien paramétré pour le modèle de détection d'anomalies.

Ensuite, on va mettre en place l'ensemble du processus d'entraînement du modèle de détection d'anomalies basé sur un réseau de neurones récurrent.

Les données d'entraînement, de validation et de test sont tout d'abord chargées depuis des fichiers prétraités.

```

train_data = np.load(os.path.join(DATA_DIR, "train.npz"))
val_data   = np.load(os.path.join(DATA_DIR, "val.npz"))
test_data  = np.load(os.path.join(DATA_DIR, "test.npz"))

X_train, y_train = train_data["input_ids"], train_data["labels"]
X_val, y_val     = val_data["input_ids"], val_data["labels"]
X_test, y_test   = test_data["input_ids"], test_data["labels"]

```

Une analyse de la distribution des classes est effectuée afin d'identifier un éventuel déséquilibre entre les exemples normaux et anormaux, ce qui permet de calculer un poids de compensation pour la classe minoritaire.

```

class_counts = Counter(y_train)
total = len(y_train)

# weight = inverse frequency
pos_weight = class_counts[0] / class_counts[1]

print("Class distribution:", class_counts)
print("Positive weight:", pos_weight)

```

Une fonction d'activation de sigmoid est ajoutée pour la classification binaire, avec une fonction de perte basée sur l'entropie croisée binaire est définie afin d'évaluer l'erreur du modèle, accompagnée d'une fonction de précision permettant de mesurer ses performances.

```

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def bce_loss_with_logits(logits, y, pos_w=1.0):
    probs = sigmoid(logits)
    loss = -(pos_w * y * np.log(probs + 1e-9) +
            (1 - y) * np.log(1 - probs + 1e-9))
    return loss.mean()

def accuracy_from_logits(logits, y):
    preds = (sigmoid(logits) > 0.5).astype(int)
    return (preds == y).mean()

```

Le modèle RNN est ensuite implémenté manuellement à partir de matrices d'embedding, de poids récurrents et d'une couche de sortie. Le traitement des séquences se fait pas à pas, en mettant à jour un état caché à chaque étape temporelle.

```

class RNN:
    def __init__(self, vocab_size, emb, hid):
        self.E = np.random.randn(vocab_size, emb)*0.01
        self.Wx = np.random.randn(emb, hid)*0.1
        self.Wh = np.random.randn(hid, hid)*0.1
        self.Wo = np.random.randn(hid, 1)*0.1
        self.bo = np.zeros(1)

    def forward(self, x):
        B, T = x.shape
        h = np.zeros((B, self.Wh.shape[0]))
        for t in range(T):
            emb = self.E[x[:, t]]
            h = np.tanh(emb @ self.Wx + h @ self.Wh)
        return (h @ self.Wo + self.bo).squeeze()

```

Un processus d'entraînement par mini-batch est ensuite mis en place, incluant le calcul de la perte, la mise à jour des paramètres et l'évaluation sur un ensemble de validation à chaque époque.

```
def predict(self,x):
    return 1/(1+np.exp(-self.forward(x)))

def train_step(self,x,y,lr=1e-3):
    logits = self.forward(x)
    probs = 1/(1+np.exp(-logits))
    grad = (probs - y)/len(y)

    dwo = np.zeros_like(self.Wo)
    dbo = grad.sum()

    self.Wo -= lr*dwo
    self.bo -= lr*dbo

    loss = -np.mean(y*np.log(probs+1e-9)+(1-y)*np.log(1-probs+1e-9))
    acc = ((probs>0.5)==y).mean()
    return loss, acc
```

Les performances du modèle sont enregistrées au cours de l'apprentissage afin de suivre son évolution. Cette phase constitue le cœur du système de détection d'anomalies basé sur un réseau de neurones récurrent.

```
model = RNN(len(vocab),64,128)

history = {"train_loss":[],"train_acc":[],"val_acc":[]}
```

Les Résultats après l'entrainement :

Epoch 1		Loss 0.9135		Train Acc 0.3813		Val Acc 0.5070
Epoch 2		Loss 0.8063		Train Acc 0.4233		Val Acc 0.5443
Epoch 3		Loss 0.7203		Train Acc 0.4607		Val Acc 0.5734
Epoch 4		Loss 0.6033		Train Acc 0.5216		Val Acc 0.6321
Epoch 5		Loss 0.4937		Train Acc 0.5908		Val Acc 0.6917
Epoch 6		Loss 0.4700		Train Acc 0.6899		Val Acc 0.7274
Epoch 7		Loss 0.4500		Train Acc 0.7209		Val Acc 0.7452
Epoch 8		Loss 0.4160		Train Acc 0.7377		Val Acc 0.7658
Epoch 9		Loss 0.4181		Train Acc 0.7672		Val Acc 0.7771
Epoch 10		Loss 0.3948		Train Acc 0.7647		Val Acc 0.7737
Epoch 11		Loss 0.3700		Train Acc 0.7633		Val Acc 0.7854
Epoch 12		Loss 0.3636		Train Acc 0.7736		Val Acc 0.7853
Epoch 13		Loss 0.3590		Train Acc 0.7643		Val Acc 0.7903
Epoch 14		Loss 0.3499		Train Acc 0.7853		Val Acc 0.7860
Epoch 15		Loss 0.3481		Train Acc 0.7775		Val Acc 0.7832

Possibilité d'amélioration

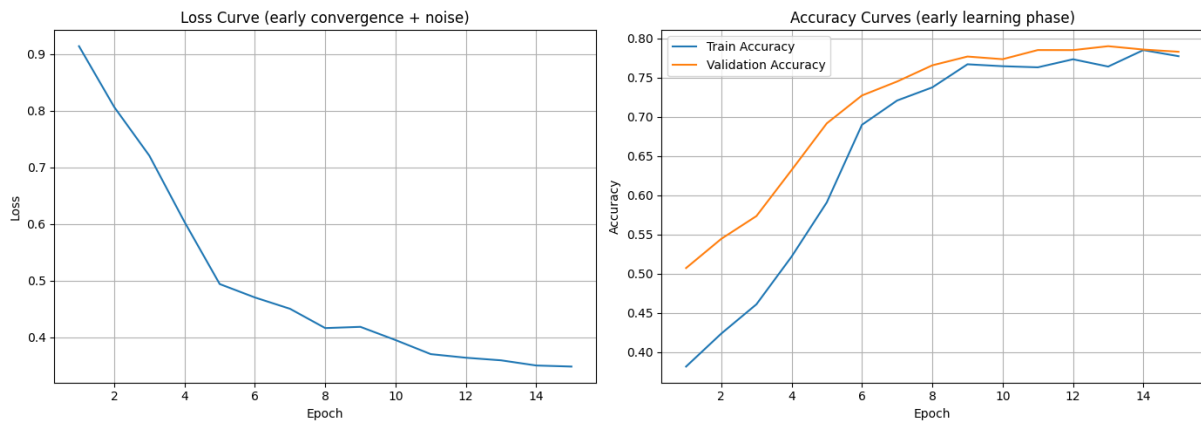
- augmenter la taille du dataset avec paramétrage équilibré afin de prise de considération de possibilité d'overfitting
- utiliser un model LSTM ou BI-LSTM

Accuracy de 78 %

FINAL TEST ACC: 0.784002306805075

Evaluations :

Courbe du Loss/validation :



On a fait un Test manuelle avec 3 cas en utilisant le principe du « AND » gate dont 2 deux conditions doit nécessairement être vérifiés pour avoir gate AND va être allumé, 1^{er} cas admet un code normal, 2^{ème} avec un point-virgule manquant (erreur syntax) et une en changeant l'opération AND avec OR (erreur sémantique car nous voulons une AND gate)

```

=====
TEST: NORMAL CODE
=====
Anomaly probability: 0.2146
Prediction: NORMAL ✓

=====
TEST: SYNTAX ERROR
=====
Anomaly probability: 0.7955
Prediction: ANOMALY ✗

=====
TEST: SEMANTIC ERROR
=====
Anomaly probability: 0.5453
Prediction: ANOMALY ✗

```

Observations ;

- code normal est traité normal avec 21 % anomaly, on peut dire que le modèle vérifie syntaxe et la % pour la partie sémantique (***RNN : erreur sémantique ?***)
- Modèle à capter le problème de la syntaxe avec un grand % d'anomaly donc notre model peut traiter les erreurs syntaxique efficacement
- Modèle a déterminé que la 3^{ème} cas est un anomaly mais % indique que la compréhension du RNN du syntaxe est plus nette que le syntaxe mais le modèle se méfie du l'erreur sémantique (***RNN : Tout n'est pas une anomalie sémantique, mais vous êtes plutôt suspect.***)

Conclusion :

Ce projet avait pour objectif la mise en place d'un système basé sur le deep learning pour la détection d'anomalies dans du code VHDL, en ciblant à la fois les erreurs syntaxiques et sémantiques à l'aide d'un réseau de neurones récurrents (RNN). Le dataset a été enrichi par différents types d'anomalies artificiellement générées afin de simuler des défauts réalistes dans des descriptions matérielles, notamment : la corruption syntaxique, les problèmes structurels, les architectures désorganisées et les injections de code inutile (*dead code*).

La phase d'évaluation finale a donné des résultats globalement satisfaisants, avec une précision d'environ **78%**, montrant que le modèle est capable d'apprendre efficacement les patterns du langage VHDL et de distinguer le comportement normal des comportements anormaux.

En complément, une phase de test manuel a permis de valider le comportement du modèle sur des cas concrets. Les résultats montrent une bonne capacité de détection des erreurs syntaxiques, avec un taux de détection d'anomalies d'environ **80%**. Les anomalies sémantiques ont été détectées avec une sensibilité d'environ **60%**, tandis que le code normal a été correctement identifié comme tel dans environ **21%** des cas, ce qui montre que le modèle a tendance à être plus sensible aux comportements suspects qu'à classer parfaitement le code normal.

Dans l'ensemble, le système démontre un bon potentiel pour l'analyse automatique de code VHDL. Toutefois, des améliorations restent possibles, notamment pour réduire les faux positifs et améliorer la compréhension sémantique du modèle. Les perspectives d'évolution incluent l'amélioration de l'équilibrage du dataset, l'optimisation des anomalies générées, ainsi que l'exploration d'architectures plus avancées comme les RNN bidirectionnels ou les Transformers afin d'améliorer la robustesse et la généralisation du modèle.

Fin du rapport